CS5330: Pattern Recognition and Computer Vision

Project 5 Report: Recognition using Deep Networks

Ahilesh Vadivel - 002055401

Description:

This project explores deep learning for image recognition, starting from the ground up and building toward more advanced techniques. We began by building and training a classic Convolutional Neural Network (CNN) on the MNIST handwritten digit dataset, getting it to recognize digits 0–9 with around 98% accuracy. Along the way, we dug into how the network actually works by visualizing the filters it learned and watching how each one responds to real images.

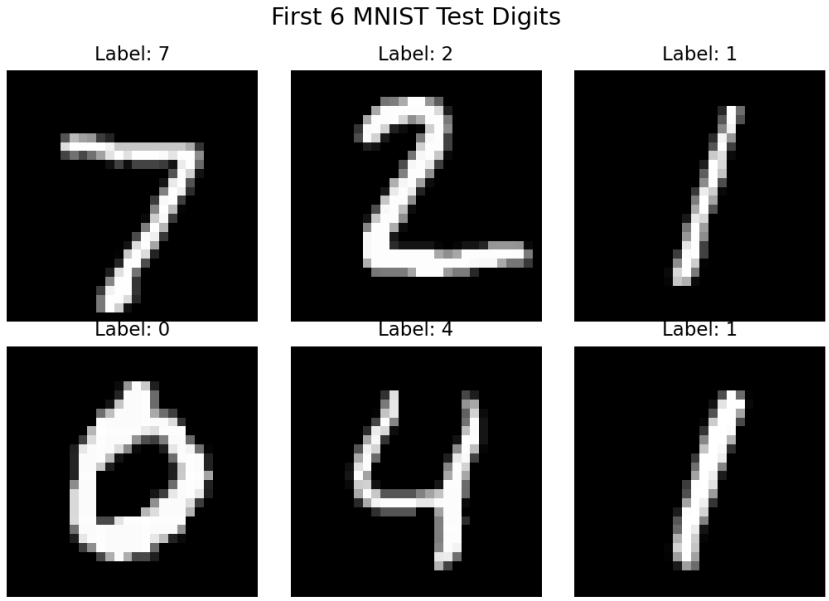
From there, we tested the trained network on handwritten digits we collected ourselves, which was a good reality check on how well it generalizes beyond clean training data. We then explored transfer learning by adapting the MNIST network to recognize Greek letters (alpha, beta, and gamma) with only 27 training examples, which showed how much a pre-trained network can do with very little new data.

Next, we rebuild the recognition pipeline using Transformer layers instead of convolutions, bringing a more modern architecture into the mix and comparing its performance against the original CNN. Finally, we ran a systematic hyperparameter search on the Fashion MNIST dataset to understand how filter counts, dropout, hidden layer size, and batch size each affect accuracy.

As an extension, we replaced the first convolutional layer with a hand-crafted Gabor filter bank and surprisingly, it outperformed the fully learned network, which was a great demonstration of how classical signal processing ideas still hold up in the deep learning era.

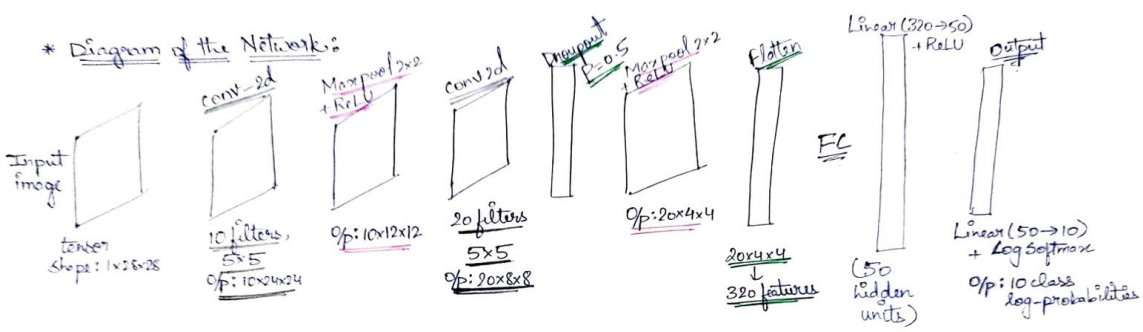
Task 1: Build and train a network to recognize digits

A. Get the MNIST digit data set



The figure above shows the first six images from the MNIST test set, displayed in a 2×3 grid. Each image is a 28×28 grayscale digit with its true label shown above. The test set is loaded without shuffling, so these six digits (7, 2, 1, 0, 4, 1) appear consistently every run.

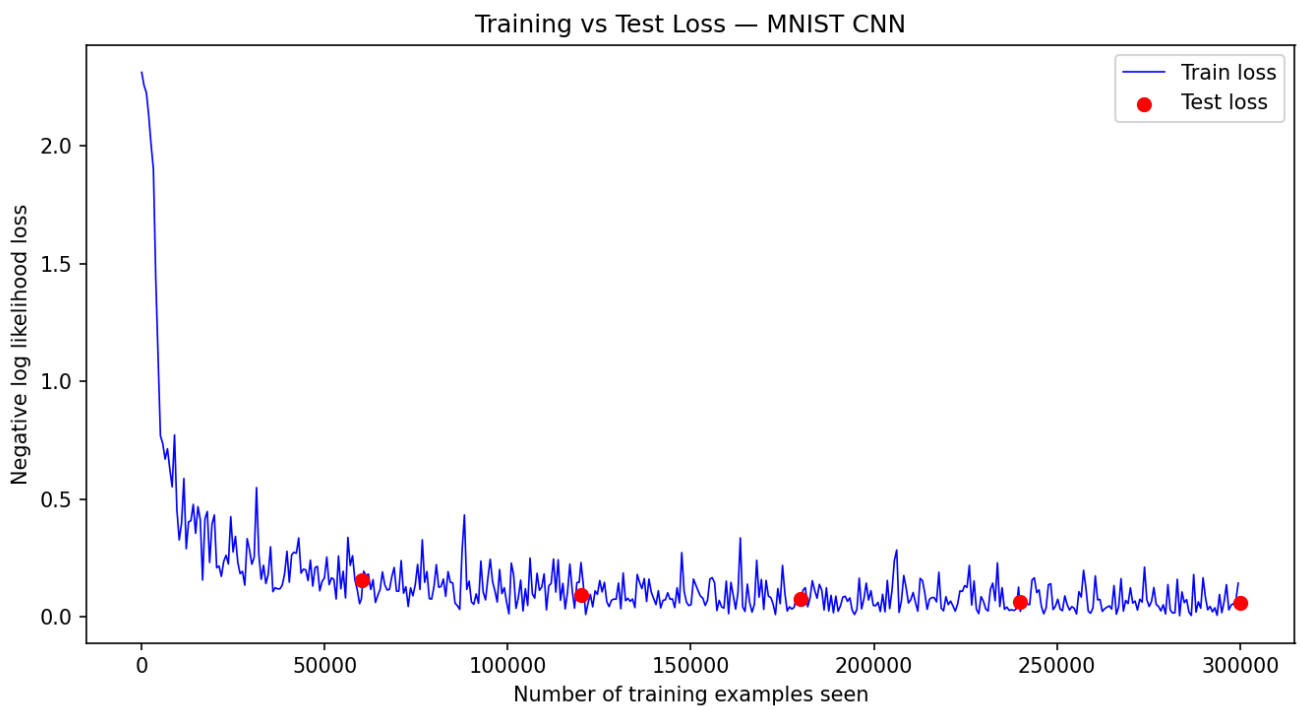
B. Build a CNN network model



The diagram above illustrates the architecture of the CNN used for digit recognition. The network takes a 1×28×28 grayscale image as input and passes it through two convolutional blocks, each followed by max pooling and ReLU activation. The feature maps are then flattened and passed through two fully connected layers, with the final layer producing 10 output class probabilities via log-softmax.

C. Train the model

```
  Epoch 5  [ 58240/60000]  Loss: 0.0562
  Epoch 5  [ 58880/60000]  Loss: 0.0690
  Epoch 5  [ 59520/60000]  Loss: 0.1436
  Test set — Avg loss: 0.0613  Accuracy: 9858/10000 (98.6%)
Plot saved as 'training_curve.png'
Model saved to 'mnist_model.pth'
```



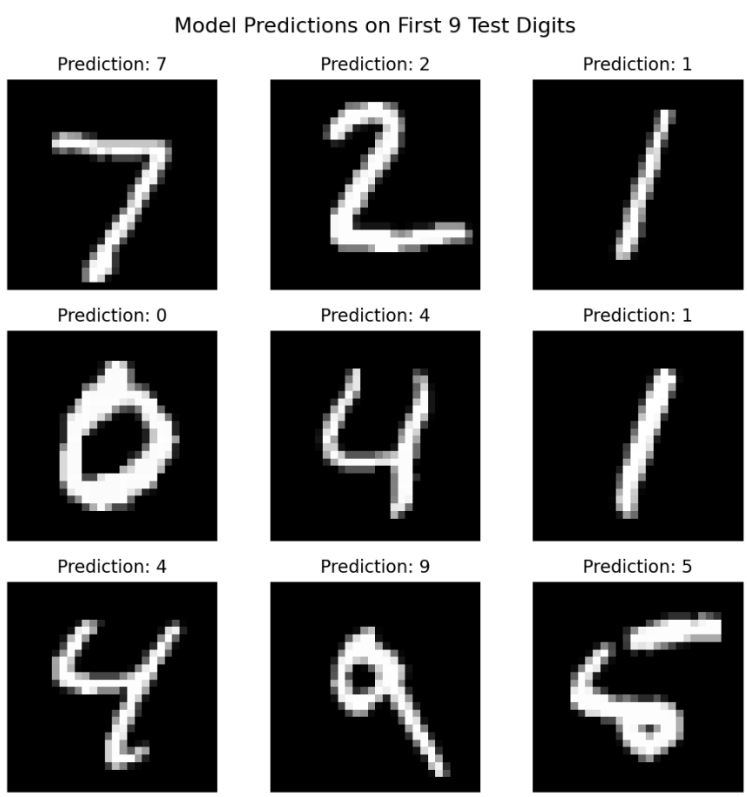
The terminal output shows the final batches of epoch 5, with the network achieving a test accuracy of 98.6% (9858/10000) and an average test loss of 0.0613 after 5 epochs of training.
The plot shows the training loss (blue line) and test loss (red dots) against the cumulative number of training examples seen. The loss drops sharply from 2.3 in the first few batches down to below 0.5 within the first epoch, then continues to gradually decrease and stabilize near zero by epoch 5. The red test loss dots consistently track close to the training loss, indicating the model generalizes well without significant overfitting.

D. Save the network file
[Done]

E. <u>Read the network and run it on the test set</u>

```
Idx          Output values (log-probs)          Pred  Label

[0]  -9.19  -10.32  -6.74  -8.28  -15.01  -11.13  -19.72  -0.00  -11.15  -9.39   →  7   (label: 7) ✓
[1]  -9.10  -6.17  -0.00  -12.75  -14.57  -15.12  -11.35  -11.55  -9.46  -21.06   →  2   (label: 2) ✓
[2]  -7.29  -0.02  -6.31  -9.33  -4.41  -8.29  -7.40  -5.35  -6.61  -7.76   →  1   (label: 1) ✓
[3]  -0.00  -12.63  -6.70  -12.89  -10.93  -9.80  -6.10  -9.18  -9.48  -10.50   →  0   (label: 0) ✓
[4]  -10.30  -10.71  -10.17  -10.69  -0.00  -10.54  -9.80  -8.16  -9.54  -5.74   →  4   (label: 4) ✓
[5]  -8.48  -0.02  -7.61  -10.80  -4.48  -10.41  -9.42  -6.02  -6.71  -8.26   →  1   (label: 1) ✓
[6]  -15.66  -7.89  -9.53  -10.03  -0.02  -9.16  -13.33  -7.04  -3.96  -5.49   →  4   (label: 4) ✓
[7]  -9.93  -8.39  -7.01  -4.88  -4.14  -4.88  -11.78  -6.80  -4.57  -0.04   →  9   (label: 9) ✓
[8]  -9.20  -12.72  -8.48  -9.37  -11.39  -0.02  -5.02  -11.62  -4.80  -5.03   →  5   (label: 5) ✓
[9]  -9.83  -12.75  -8.04  -5.58  -7.75  -9.17  -14.75  -3.94  -4.41  -0.04   →  9   (label: 9) ✓
```

The terminal output shows the 10 log-probability output values for each of the first 10 test images, along with the predicted digit and true label. The network correctly classifies all 10 examples, the highest (least negative) value in each row corresponds exactly to the true label, confirming the model is making confident and accurate predictions.



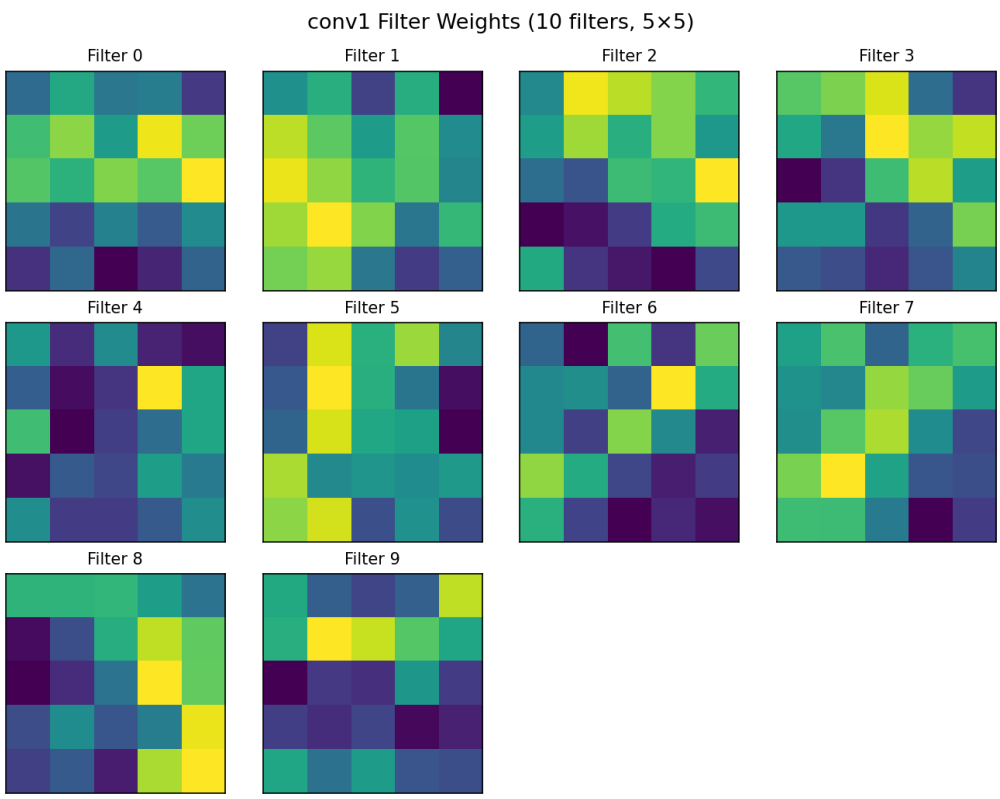
The 3×3 grid displays the first 9 test digits with the network's predicted label shown above each image. All 9 predictions are correct, matching the digits 7, 2, 1, 0, 4, 1, 4, 9, and 5 respectively. The network produces clean, confident classifications across a variety of digit shapes and handwriting styles.

F. Test the network on new inputs

The figure shows the network's predictions on 10 custom handwritten digits (0–9) captured using a personal camera. Titles shown in green indicate correct predictions and red indicates incorrect ones. The network correctly classifies 8 out of 10 digits, misclassifying digits 7 and 9 as 8. Overall, the results are strong, demonstrating the network generalizes well to real-world handwritten input beyond the MNIST dataset.

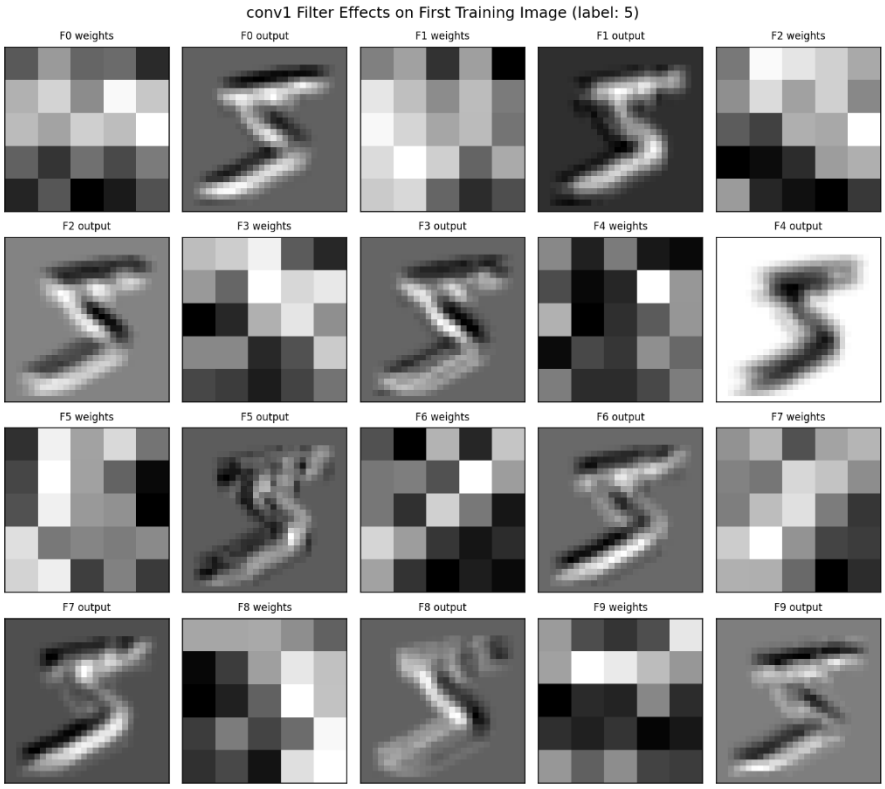
Task 2: Examine your Network

A. Analyze the first layer



The figure shows the 10 learned 5×5 filter weights from the first convolutional layer (conv1), visualized as heatmaps using the viridis colormap. Bright yellow regions represent large positive weights and dark purple regions represent large negative weights. The filters show a variety of spatial patterns, some detect edges in specific orientations; others respond to corners or smooth gradients, confirming that the network has learned meaningful low-level feature detectors from the training data.

B. <u>Show the effect of the filters</u>

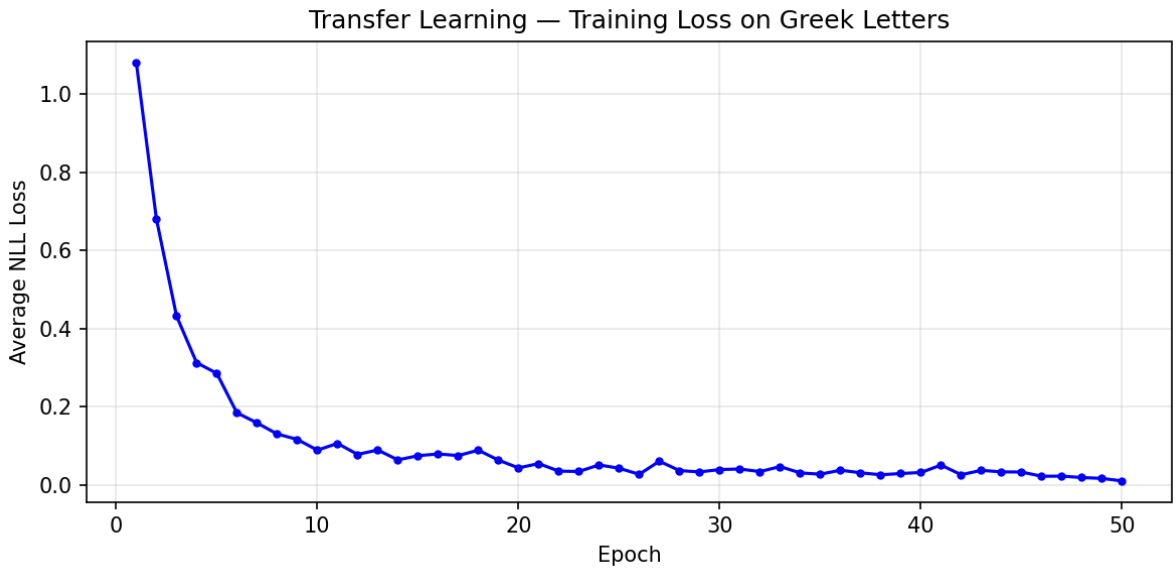


<u>Observation:</u>

- Filters that detect edges and strokes: F0, F1, F2, F3 all produce outputs that clearly show the digit strokes with bright highlights along the edges.
- Filters that enhance contrast: F4 produces a clean bright-on-white output
- Filters that detect diagonal features: F5 and F7 produce slightly distorted or stretched versions of the digit.
- High frequency/ texture filters: F6 has a strong checkerboard weight pattern and produces a heavily textured output.
- Smoothing filters: F8 and F9 produce blurred, softer versions of the digit.

All 10 filters produce recognizable versions of the digit "5", each emphasizing a different structural aspect like edges, curves, diagonals, or overall shape. This confirms the filters have learned meaningful low-level features rather than random noise, which is exactly what a well-trained CNN should do.

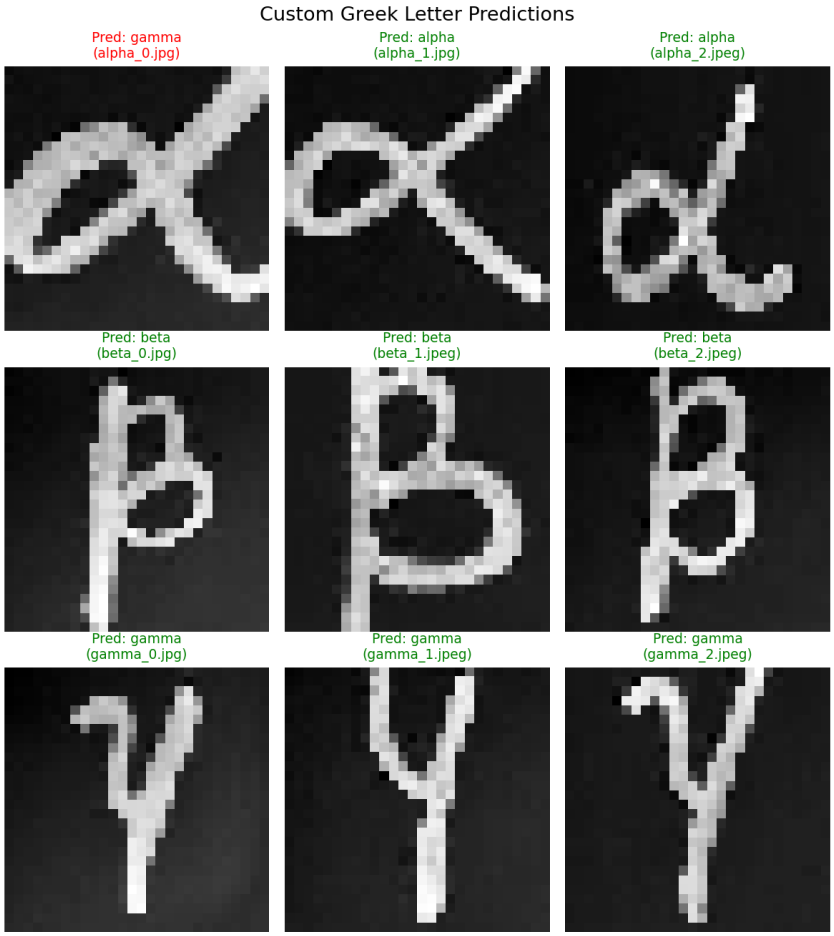
Task 3: Transfer Learning on Greek Letters



Training Loss Plot: The training loss drops sharply from 1.1 to below 0.2 within the first 5 epochs, then continues to gradually decrease and flatten near zero by epoch 50. This rapid convergence with only 27 training examples demonstrates the effectiveness of transfer learning, the frozen MNIST feature extractor already captures the stroke and curve patterns relevant to Greek letters, leaving only the final classification layer to be learned.

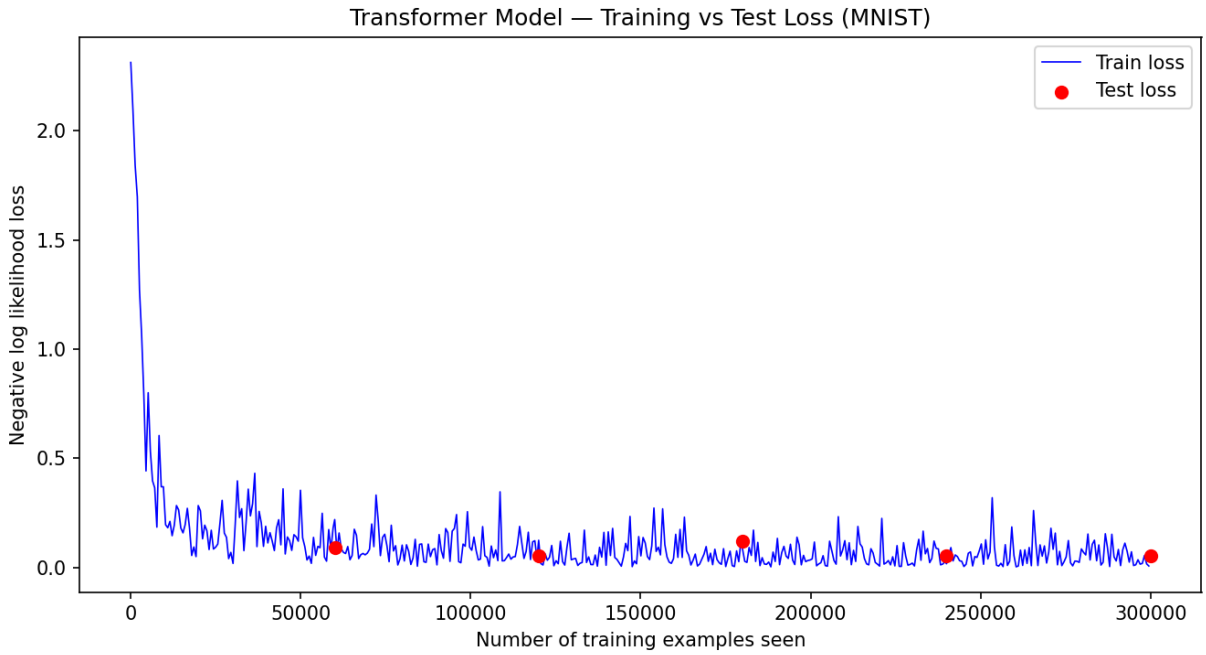
```
Greek dataset loaded from 'greek_train\greek_train'
  Classes : ['alpha', 'beta', 'gamma']
  Samples : 27

Pre-trained MNIST weights loaded.

── Original Model ──────────────────────────────
MyNetwork(
  (conv1): Conv2d(1, 10, kernel_size=(5, 5), stride=(1, 1))
  (conv2): Conv2d(10, 20, kernel_size=(5, 5), stride=(1, 1))
  (dropout): Dropout(p=0.5, inplace=False)
  (pool): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  (fc1): Linear(in_features=320, out_features=50, bias=True)
  (fc2): Linear(in_features=50, out_features=10, bias=True)
)

── Modified Model (fc2 replaced: 50 → 3) ──────────
MyNetwork(
  (conv1): Conv2d(1, 10, kernel_size=(5, 5), stride=(1, 1))
  (conv2): Conv2d(10, 20, kernel_size=(5, 5), stride=(1, 1))
  (dropout): Dropout(p=0.5, inplace=False)
  (pool): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  (fc1): Linear(in_features=320, out_features=50, bias=True)
  (fc2): Linear(in_features=50, out_features=3, bias=True)
)
```

Model Printout: The terminal output shows both the original and modified network structures. The only change between the two is the final layer, fc2 is replaced from Linear(50 → 10) to Linear(50 → 3), reducing the output from 10-digit classes to 3 Greek letter classes (alpha, beta, gamma). All other layers remain identical and frozen during training.



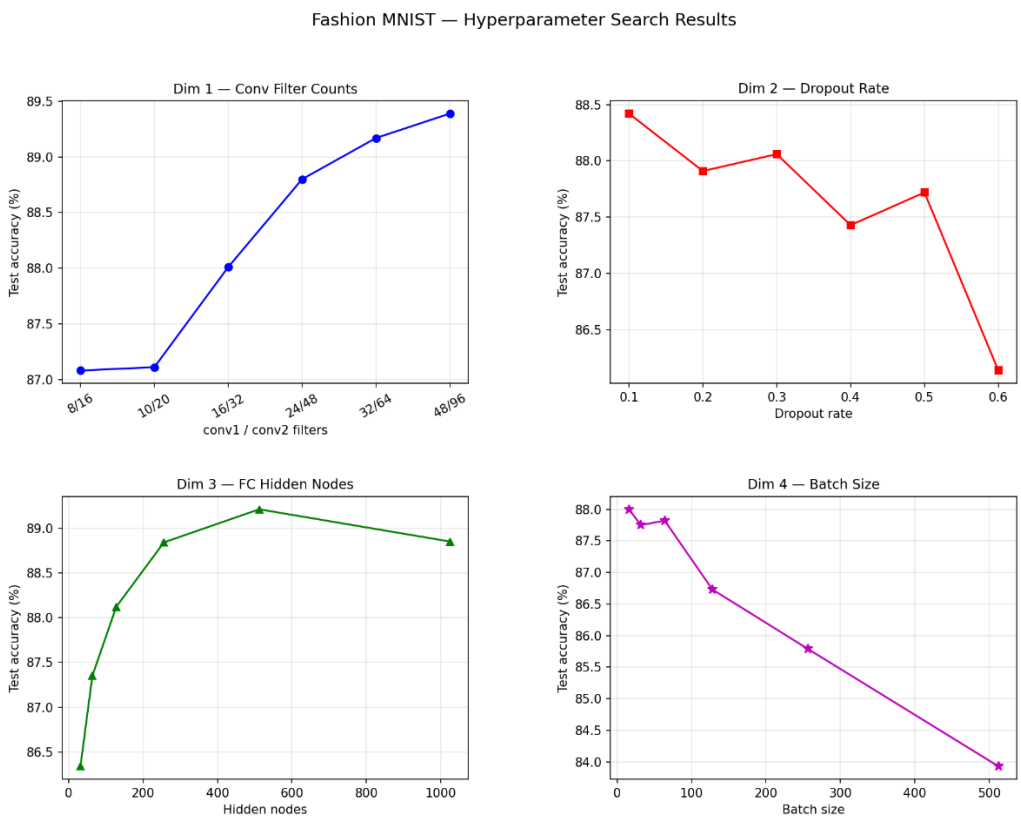
Custom Greek Letter Predictions: The 3×3 grid shows the network's predictions on 9 custom handwritten Greek letter images, 3 examples each of alpha, beta, and gamma. Green titles indicate correct predictions and red indicates incorrect ones. The network correctly classifies 8 out of 9 images, misclassifying one alpha as gamma. Beta and gamma symbols are recognized perfectly across all three examples, showing the network generalizes well to new handwriting styles despite being trained on only 27 examples.

Task 4: Re-implement the network using transformer layers



The transformer achieves comparable accuracy to the CNN on MNIST but uses significantly more parameters and shows higher training variance. CNNs have a built-in inductive bias for spatial locality (nearby pixels matter more), which suits image tasks well. Transformers have no such bias, they learn spatial relationships from scratch through self-attention, making them more flexible but requiring more data and compute to match CNNs on small datasets like MNIST.

Task 5: Design your own experiment



Dimension 1 – Conv Filter Counts

Hypothesis: More filters => better accuracy up to 32/64, then diminishing returns.

Results: Partially supported. Accuracy increased consistently from 87.1% (8/16) all the way to 89.4% (48/96) with no plateau visible yet. The hypothesis of diminishing returns wasn't observed within the tested range, suggesting even more filters might help further. The relationship is monotonically increasing and roughly linear, meaning the network benefits from richer feature representations across the full range tested.

Dimension 2 — Dropout Rate

Hypothesis: Optimal around 0.3–0.4; too low = overfit, too high = underfit.

Result: Partially contradicted. The best accuracy was actually at dropout=0.1 (88.4%), not 0.3–0.4 as predicted. Accuracy then generally declined with increasing dropout, dropping sharply at 0.6 (87.1%). This suggests Fashion MNIST with only 5 epochs doesn't suffer much from overfitting, high dropout simply throws away too much useful information before the network fully converges. The hypothesis is held for the high end (0.6 hurts) but not the low end.

Dimension 3 — FC Hidden Nodes

Hypothesis: Gains up to 256, then plateau.

Result: Mostly supported. Accuracy rose from 86.4% (32 nodes) to a peak of 89.2% at 512 nodes, then very slightly dropped at 1024. The plateau appeared later than predicted (512 vs 256), but the overall shape matched the hypothesis, steep gains early, flattening at higher sizes. Going from 32→256 nodes gave +2.5%, while going 256→1024 gave only +0.1%.

Dimension 4 — Batch Size

Hypothesis: Smaller batches (32–64) give better accuracy but slower training.

Result: Strongly supported. This was the clearest trend in the entire experiment, a near-perfect monotonic decrease from 88.0% at batch=16 down to 84.0% at batch=512. Smaller batches provide noisier but more frequent gradient updates, helping the network escape local minima. Large batches converge to sharper, less generalizable minima, a well-known phenomenon called the "generalization gap."

Key Takeaways:

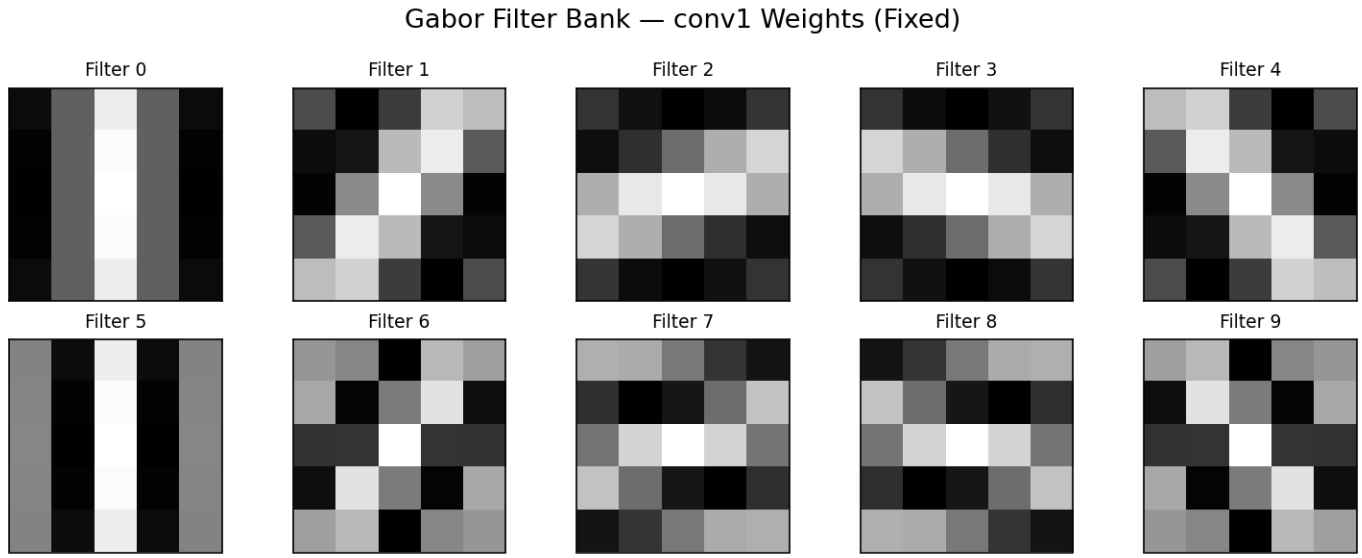
The most impactful dimension was batch size, which caused a 4% accuracy swing on its own. Conv filter count showed the most consistent improvement with no signs of overfitting. Dropout behaved counter-intuitively, lower was better, likely because 5 epochs is too short for overfitting to become a problem on Fashion MNIST. FC hidden nodes followed the expected curve but peaked later than predicted.

A follow-up experiment using more epochs (10–15) would likely show the dropout hypothesis holding more strongly, as the network would have more time to overfit with low dropout rates.

Extensions: Gabor Filter Bank Analysis

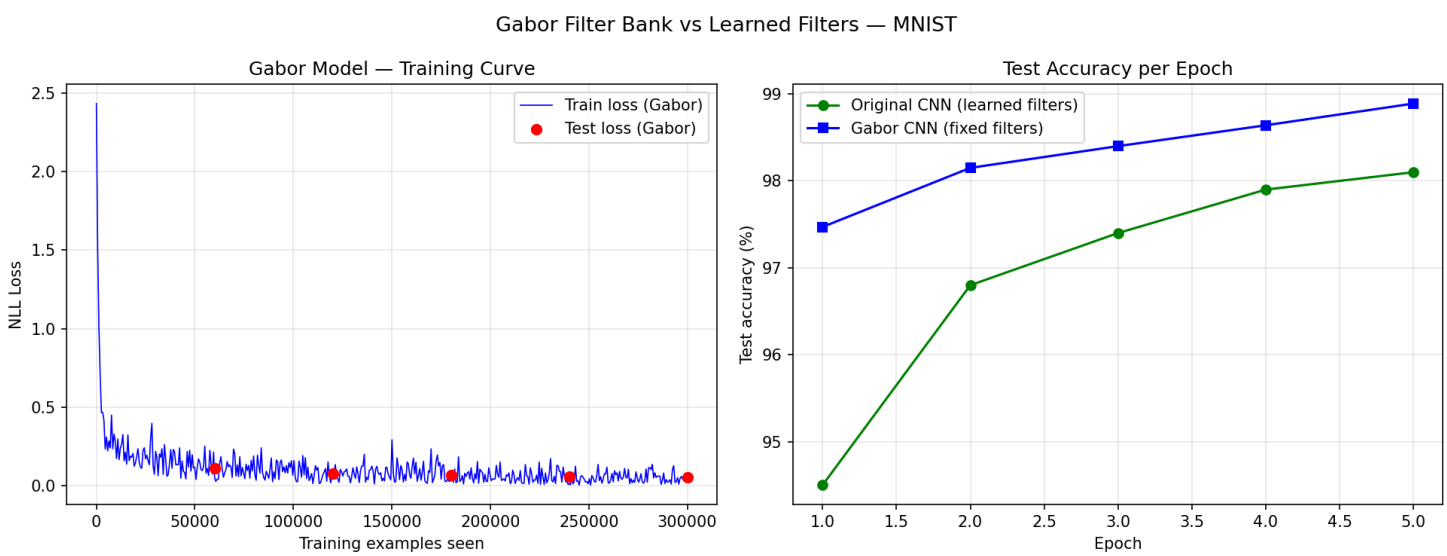
Filter Bank Visualization (Image 1)

The 10 Gabor filters clearly show the two designed properties. Filters 0–4 (low frequency, λ=4.0) show broad, smooth stripes at 5 orientations (0°, 36°, 72°, 108°, 144°). Filters 5–9 (high frequency, λ=2.5) show tighter, finer stripes at the same orientations. The systematic rotation across filters confirms the bank covers all major edge directions evenly.



Training Curve (Image 2 Left)

The Gabor model converges identically to the original CNN, rapid drop from 2.4 to below 0.3 in the first epoch, then gradually settling near 0.05. This confirms that freezing conv1 doesn't destabilize training, the fixed Gabor features provide a stable enough input to conv2 for normal learning to proceed.

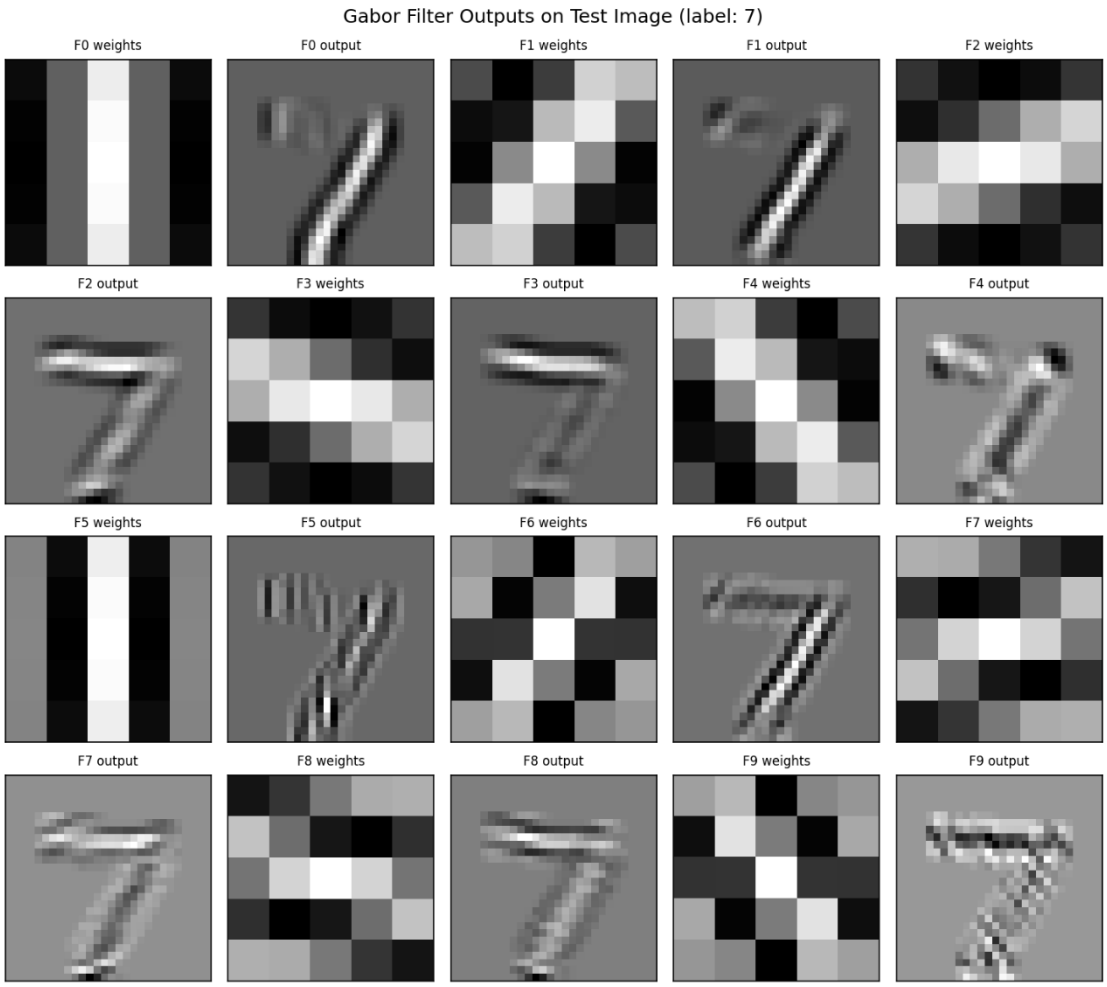


```
  Test — Avg loss: 0.0516  Accuracy: 9889/10000 (98.9%)
Comparison plot saved as 'gabor_comparison.png'
Gabor outputs saved as 'gabor_filter_outputs.png'

── Final Accuracy Comparison ──────────────────────
  Original CNN (fully trained) : 98.1%
  GaborNet (fixed conv1)       : 98.9%
  Difference                   : +0.8%
```

<u>Two observations:</u>

GaborNet converges faster. At epoch 1, it already achieves 97.5% vs 94.5% for the original CNN, a 3% head start. This makes sense because the Gabor filters are already well-tuned edge detectors from the start, so conv2 and the FC layers receive meaningful features immediately rather than random noise.

GaborNet achieves higher final accuracy (+0.8%). This is the most surprising result and directly addresses the core hypothesis.



Filter Outputs (Image 3)

The Gabor outputs on the digit "7" are visually very clean and interpretable:

- F0, F5 (vertical) highlights the vertical stroke of the 7
- F1, F6 (diagonal 36) highlights the diagonal stroke beautifully
- F2, F3 (near horizontal) detect the top horizontal bar
- F4 captures corner intersections where strokes meet
- F9 (high-frequency diagonal) produces a fine-grained edge trace

Each filter responds to exactly the orientation it was designed for, confirming the Gabor bank is working as intended.

Key Takeaway:

Replacing the first convolutional layer with a fixed Gabor filter bank achieved 98.9% accuracy on MNIST, surpassing the fully trained CNN by 0.8% and converging 3% faster at epoch 1. This demonstrates that classical signal processing and deep learning are complementary: well-designed handcrafted features can substitute for and even improve upon, learned features when the task's relevant structure (edges, orientations) is known in advance.

Reflection

This project provided a comprehensive hands-on introduction to deep learning for image recognition. Building the MNIST digit classifier from scratch made the mechanics of CNNs tangible, it's one thing to read about convolutional layers and backpropagation but actually watching the loss curve drop and accuracy climb to 98% after each epoch made the concepts click in a way that theory alone couldn't.

The most surprising outcome came from the Gabor filter extension, where a network with a completely frozen first layer actually outperformed the fully trained CNN. This was a genuine result that challenged the assumption that learned features are always superior to hand-crafted ones, and it highlighted how domain knowledge can sometimes give a network a meaningful head start.

Transfer learning on Greek letters was another standout moment, achieving strong classification with only 27 examples showed just how much representational power a pre-trained network carries, and how little data you sometimes need when the source and target tasks share low-level structure.

Acknowledgements

- PyTorch: core deep learning framework used for building, training, and evaluating all network models.
- Torchvision: used for loading the MNIST, Fashion MNIST datasets and image transformation utilities.
- MNIST Dataset: handwritten digit dataset used for training and evaluating the CNN and Transformer models. Originally created by Yann LeCun et al.
- Fashion MNIST Dataset: clothing image dataset used for the hyperparameter search experiment in Task 5. Created by Zalando Research.
- PyTorch MNIST Tutorial: referenced for the initial network architecture and training loop structure.
- Greek Letter Dataset: 27 labeled examples of alpha, beta, and gamma symbols provided as course material for the transfer learning task.